Lab 5: Aspect-Oriented Programming (AOP)

Modularize logging, execution timing and security auditing using Spring AOP

Continue project from Lab 4.

What you will learn

- Add spring-boot-starter-aop dependency to pom.xml file
- Master the core AOP terminology: Aspect, Join Point, Pointcut and Advice.
- Create a reusable LoggingAspect using @Before, @AfterReturning and @AfterThrowing.
- Extract authenticated user identity from Spring Security inside an Aspect to build an audit trail.
- Create a PerformanceAspect with @Around and ProceedingJoinPoint to track method execution duration.
- Observe how AOP proxies intercept calls at runtime without modifying a single line of business code.

Clean Separation of Concerns

By applying Spring AOP, business services contain only business logic.

Cross-cutting concerns like logging, execution timing and security audits are extracted into standalone Aspect classes, intercepting service calls seamlessly via runtime proxies.

✓ Step 1. Add Spring AOP to the pom.xml file

Open the pom.xml file and add spring-boot-starter-aspectj inside the <dependencies> block.

This provides AspectJ annotations and runtime proxy support.

pom.xml (add inside <dependencies>)

```
<!-- Spring Boot Starter AOP: AspectJ + Spring AOP Proxies -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-aspectj</artifactId>
</dependency>
```

- spring-boot-starter-aspectj: brings in AspectJ Weaver and configures CGLIB proxying automatically.
- Reload Maven dependencies in your IDE after updating pom.xml.

✓ Step 2. Project structure overview

Review your updated project layout.

We introduce a new package aspect/ containing LoggingAspect.java and PerformanceAspect.java.

All controllers, services, repositories and entities remain completely untouched.

Project package structure

```
src/main/java/com/example/todolist/
├── aspect/                          # AOP aspects package for cross-cutting concerns
│   ├── LoggingAspect.java           # method tracing, arguments and security audit
│   └── PerformanceAspect.java       # method execution duration with @Around
├── config/
│   └── SecurityConfig.java
├── controller/
│   ├── TodoController.java
│   └── api/v1/
│       └── TodoRestController.java
├── dto/
│   └── TodoRequest.java
├── exception/
│   └── TodoNotFoundException.java
├── model/
│   └── Todo.java
│   └── User.java
├── repository/
│   └── TodoRepository.java
│   └── UserRepository.java
├── service/
│   └── CustomUserDetailsService.java
│   └── TodoService.java
└── TodolistApplication.java
```

✓ Step 3. Create LoggingAspect with security auditing

Create package aspect and add LoggingAspect.java.

We define a pointcut matching all methods in TodoService, log method calls with the current authenticated username and capture returns and thrown exceptions.

src/main/java/com/example/todolist/aspect/LoggingAspect.java (Part 1 - Pointcut and @Before)

```
package com.example.todolist.aspect;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.AfterReturning;
import org.aspectj.lang.annotation.AfterThrowing;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.aspectj.lang.annotation.Pointcut;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;
import java.util.Arrays;

@Aspect
@Component
public class LoggingAspect {

    private static final Logger log = LoggerFactory.getLogger(LoggingAspect.class);

    @Pointcut("execution(* com.example.todolist.service.TodoService.*(..))")
    public void todoServiceMethods() {
    }

    @Before("todoServiceMethods()")
    public void logBefore(JoinPoint jp) {
        String user = getCurrentUser();
        String method = jp.getSignature().getName();
        Object[] args = jp.getArgs();
        log.info("[AOP BEFORE] User: '{}' -> calling method: {}({})", user, method,
Arrays.toString(args));
    }
```

LoggingAspect.java (continued - Returns, Exceptions, and Security Context)

```
    @AfterReturning(pointcut = "todoServiceMethods()", returning = "result")
    public void logAfterReturning(JoinPoint jp, Object result) {
        String method = jp.getSignature().getName();
        log.info("[AOP AFTER-RETURNING] Method: {} -> returned: {}", method, result);
    }

    @AfterThrowing(pointcut = "todoServiceMethods()", throwing = "ex")
    public void logAfterThrowing(JoinPoint jp, Throwable ex) {
        String method = jp.getSignature().getName();
        log.error("[AOP AFTER-THROWING] Method: {} -> threw exception: {}", method,
ex.getMessage());
    }

    private String getCurrentUser() {
        Authentication auth = SecurityContextHolder.getContext().getAuthentication();
        return (auth != null && auth.isAuthenticated()) ? auth.getName() : "anonymous";
    }
}
```

- @Aspect and @Component: marks this class as an AOP aspect and registers it as a Spring bean.
- @Pointcut: defines a reusable expression targeting every method inside TodoService regardless of return type or parameters.
- @Before: intercepts calls before target execution, inspecting method name and arguments via JoinPoint.
- @AfterReturning / @AfterThrowing: captures returned objects or uncaught exceptions.
- getCurrentUser(): extracts username from SecurityContextHolder, combining Spring Security (Lab 4) with AOP auditing.

✓ Step 4. Create PerformanceAspect with @Around Advice

Add PerformanceAspect.java in package aspect.

We use @Around advice to intercept service calls, measure elapsed execution time and log the duration.

src/main/java/com/example/todolist/aspect/PerformanceAspect.java

```
package com.example.todolist.aspect;

import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class PerformanceAspect {

    private static final Logger log = LoggerFactory.getLogger(PerformanceAspect.class);

    @Around("execution(* com.example.todolist.service.TodoService.*(..))")
    public Object measureExecutionTime(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.currentTimeMillis();

        Object result = pjp.proceed();

        long duration = System.currentTimeMillis() - start;
        String method = pjp.getSignature().toShortString();
        log.info("[AOP PERFORMANCE] {} completed in {} ms", method, duration);

        return result;
    }
}
```

- @Around: surrounds the target method, allowing custom logic before and after execution.
- ProceedingJoinPoint: represents the intercepted method call; calling pjp.proceed() executes the actual service method.
- duration: measures execution elapsed time in milliseconds and outputs to console logs.
- return result: must return the value obtained from pjp.proceed(), otherwise the caller receives null.

✓ Step 5. Test AOP logging and timing in the browser

Run the application and observe the console while performing actions on the web interface.

1. Start TodolistApplication.java and open your browser at http://localhost:8080/todos
2. Log in with user credentials ('user' / 'password123').
3. Observe the console log: notice [AOP BEFORE] User: 'user' -> calling method: findAll([]).
4. Notice [AOP PERFORMANCE] TodoService.findAll() completed in X ms.
5. Notice [AOP AFTER-RETURNING] Method: findAll -> returned: [Todo{id=...}].
6. Create a new todo 'Learn Spring AOP' via the HTML form: observe [AOP BEFORE] User: 'user' -> calling method: create(...) followed by duration and return value.

✓ Step 6. Test AOP interception and exception handling via RESTful API

Use Postman to call RESTful API endpoints and verify AOP interception on both successful calls and errors.

1. Open Postman, configure Basic Auth with 'admin' / 'admin123' and send GET http://localhost:8080/api/v1/todos
2. Check console: see [AOP BEFORE] User: 'admin' -> calling method: findAll([]).
3. Send GET http://localhost:8080/api/v1/todos/99999 with an ID that does not exist in MySQL.
4. Observe the console: [AOP BEFORE] User: 'admin' -> calling method: findById([99999]).
5. Notice [AOP AFTER-THROWING] Method: findById -> threw exception: Todo not found with id: 99999.
6. Verify that TodoRestController returns HTTP 404 NOT_FOUND JSON as configured in Lab 2, while the AOP aspect cleanly logged the exception!

✓ Step 7. Summary

- The pom.xml file includes spring-boot-starter-aspectj dependency.
- LoggingAspect is marked with @Aspect and @Component and targets TodoService methods.
- LoggingAspect successfully logs the authenticated username, method name and arguments using @Before.
- LoggingAspect logs normal return values with @AfterReturning and thrown exceptions with @AfterThrowing.
- PerformanceAspect wraps service methods using @Around and logs execution duration in milliseconds.
- No modifications were made to TodoService, TodoRepository or either controller.